

Q7. Write short notes on transactions and schedules

A **transaction** is a sequence of database operations that are treated as a single logical unit of work. Transactions help ensure the integrity of data, even when multiple operations are executed simultaneously or the system crashes. To maintain data consistency, transactions must satisfy the **ACID** properties:

- **Atomicity** – Ensures that either all operations in the transaction are executed, or none are.
- **Consistency** – Guarantees that a transaction brings the database from one valid state to another.
- **Isolation** – Each transaction is executed independently of others.
- **Durability** – Once a transaction commits, its effects persist even in case of failure.

A **schedule** is the order in which operations from multiple transactions are interleaved. Schedules are of two main types:

- **Serial Schedule**: Executes one transaction at a time, ensuring complete isolation.

- **Concurrent Schedule:** Interleaves operations of different transactions to improve throughput and system efficiency.

Concurrency control techniques like **locking**, **timestamp ordering**, and **serializability testing** are used to ensure correctness in concurrent schedules.

Q8. Explain various features of relational databases

Relational databases organize data into structured tables (relations). Key features include:

1.
Data in Tables: Data is stored in rows (tuples) and columns (attributes), forming relations.
2.
SQL Support: SQL (Structured Query Language) provides a standardized way to query and manipulate relational data.
3.
Normalization: Reduces data redundancy and ensures data integrity by dividing data into well-structured related tables.
4.
Data Integrity: Enforced using constraints like **Primary Keys**, **Foreign Keys**, **CHECK**, and **NOT NULL**.

5.

ACID Properties: Ensure reliable transaction processing (Atomicity, Consistency, Isolation, Durability).

6.

Data Independence: Allows schema changes without affecting the application.

7.

Relationships and Keys: Supports 1:1, 1:N, and M:N relationships via keys.

8.

Concurrent Access and Security: Offers multi-user access with transaction isolation, role-based permissions, and access control.

These features make relational databases highly reliable and widely used in business, banking, and web applications.

Q9. Write the procedure of testing for serializability. Support your answer with suitable example

Serializability is the standard for determining the correctness of concurrent schedules. A schedule is **conflict serializable** if its result is the same as some serial execution of the same transactions.

Steps for Testing Conflict Serializability:

1.

Identify Transactions and Operations:

List all read (R) and write (W) operations in the schedule.

2.

Detect Conflicts:

Conflicts occur when two operations:

- Are from different transactions,
- Access the same data item,
- At least one is a write.

3.

Build Precedence Graph (Serialization Graph):

- Create a node for each transaction.

- For each conflict, draw a directed edge from the earlier transaction to the later one.

4.

Check for Cycles:

- If the graph has no cycles, the schedule is conflict serializable.
- If cycles exist, the schedule is **not** serializable.

Example:

Transactions:

T1: R(A), W(A)

T2: R(A), W(A)

Schedule:

T1: R(A)

T2: R(A)

T1: W(A)

T2: W(A)

Conflicts:

- T1 W(A) $\bowtie$ T2 W(A): T1 $\rightarrow$ T2
- T2 R(A) $\bowtie$ T1 W(A): T2 $\rightarrow$ T1

Graph: T1 $\rightarrow$ T2 and T2 $\rightarrow$ T1 $\bowtie$ **Cycle exists**, so the schedule is **not serializable**.

Q10. What are the properties of a transaction?

The properties of a database transaction are defined by the **ACID** principle:

1.

Atomicity:

- A transaction must be fully completed or not executed at all.

- If a failure occurs, any partial changes are undone.

2.

Consistency:

- Ensures the database remains in a valid state before and after the transaction.
- Preserves integrity constraints (e.g., foreign keys, balance totals).

3.

Isolation:

- Concurrent transactions are executed as if they were sequential.
- Intermediate states are not visible to other transactions.

4.

Durability:

- Once a transaction is committed, its changes are permanent.
- Survives system crashes through logging and recovery mechanisms.

Example:

In a bank transfer of ₹1000 from Account A to B:

- Atomicity ensures both debit and credit occur together.
- Consistency ensures total balance is unchanged.
- Isolation prevents interference from other transfers.
- Durability guarantees that after commit, the changes persist.

Q11. Distinguish between deferred and immediate database modification with example

Feature	Deferred Modification	Immediate Modification
When update is applied	After the transaction reaches commit	As soon as the update statement executes
Undo required?	No (changes not yet applied)	Yes (to roll back partial updates)
Redo required?	Yes (to apply updates after failure)	Yes (for committed data restoration)
Logging type	Only redo logs	Both undo and redo logs
System complexity	Simpler	More complex
Recovery from failure	Easier, as changes are pending	Needs both undo and redo operations